Using BigQuery Embeddings in a RAG Architecture

Lab · 1 hour · No cost · Intermediate

Rate Lab

This lab may incorporate AI tools to support your learning.

Overview

In this lab, you will load a dataset of text into BigQuery, and use BigQuery's ability to create text embeddings for each chunk of text.

You will then implement a Retrieval-Augmented Generation (RAG) architecture, which is used to generate an embedding of a user's query, find the most similar embeddings of data from our text dataset, and use those corresponding chunks of text to provide a language model relevant info to use when generating its response to the user's query.

What are embeddings?

Like all machine learning models, large language models must convert all inputs to numerical representations in order to work with them.

Part of the process of training a large language model includes guiding it to learn to represent **tokens** numerically. Tokens may be words like "glass" or pieces of words, like "un", "break" and "able" from the word "unbreakable". Language models learn to represent an individual token as a **vector embedding**, or string of numbers like [0.12232, 0.23442, 0.23242, ...] (using the embedding model `text-embedding-004` this would be a list 768 numbers long, so we'll stop there). We would refer to the length of this array as the number of *dimensions* of the embedding.

Multiple tokens can be combined into a single embedding, still with the same 768 dimensions. So that is why we can also generate embeddings for sentences, paragraphs, or even pages of text. Note that there is a limit, which in the case of `text-embedding-004` is a token limit of 2,048 tokens before input text is truncated.

These vector embeddings represent coordinates in an *embedding space*, with similar tokens appearing nearby one another. So just like in Algebra class, where you may have calculated the distance between two points of x and y coordinates like (0, 0) and (3, 3), we can calculate the distance between text embeddings. In this way, we can determine which content a language model considers to have similar meaning by the relatively smaller distances between the coordinates represented by their embeddings.

Objective

Create text embeddings within BigQuery and use them in a RAG architecture to inform a language model's responses.

What you will learn

In this lab, you learn how to:

- Connect BigQuery to a Vertex AI embedding model.
- Use this model to create text embeddings and store them in a BigQuery table.
- Use this model to generate an embedding for a query and perform a similarity search on the embeddings using SQL in BigQuery.
- Code a Q&A system that retrieves relevant information to a query and uses Google Gemini to answer questions about it.

Setup and Requirements

Before you click the Start Lab button

Read these instructions. Labs are timed and you cannot pause them. The timer, which starts when you click **Start Lab**, shows how long Google Cloud resources will be made available to you.

This Qwiklabs hands-on lab lets you do the lab activities yourself in a real cloud environment, not in a simulation or demo environment. It does so by giving you new, temporary credentials that you use to sign in and access Google Cloud for the duration of the lab.

What you need

To complete this lab, you need:

- Access to a standard internet browser (Chrome browser recommended).
- Time to complete the lab.

Note: If you already have your own personal Google Cloud account or project, do not use it for this lab.

Note: If you are using a Pixelbook, open an Incognito window to run this lab.

How to start your lab and sign in to the Google Cloud console

1. Click the **Start Lab** button. If you need to pay for the lab, a dialog opens for you to select your payment method. On the left is the Lab Details pane with the following:

    - The Open Google Cloud console button
    - Time remaining
    - The temporary credentials that you must use for this lab
    - Other information, if needed, to step through this lab

2. Click **Open Google Cloud console** (or right-click and select **Open Link in Incognito Window** if you are running the Chrome browser).

    The lab spins up resources, and then opens another tab that shows the Sign in page.

    Tip: Arrange the tabs in separate windows, side-by-side.

    > **Note:** If you see the **Choose an account** dialog, click **Use Another Account**.

3. If necessary, copy the **Username** below and paste it into the **Sign in** dialog.

    ```
    student-02-5c8022007301@qwiklabs.net
    ```

    You can also find the Username in the Lab Details pane.

4. Click **Next**.

5. Copy the **Password** below and paste it into the **Welcome** dialog.

    ```
    3e9E0BP9w7zR
    ```

    You can also find the Password in the Lab Details pane.

6. Click **Next**.

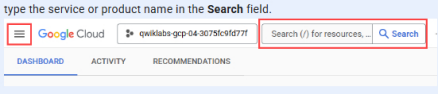
    > **Important:** You must use the credentials the lab provides you. Do not use your Google Cloud account credentials.

    > **Note:** Using your own Google Cloud account for this lab may incur extra charges.

7. Click through the subsequent pages:

    - Accept the terms and conditions.
    - Do not add recovery options or two-factor authentication (because this is a temporary account).
    - Do not sign up for free trials.

After a few moments, the Google Cloud console opens in this tab.

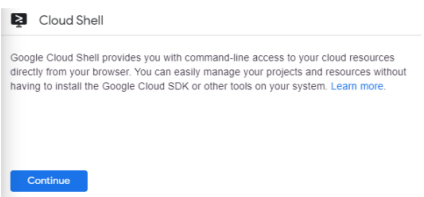
> **Note:** To access Google Cloud products and services, click the **Navigation menu** or type the service or product name in the **Search** field.
>
> 
>

Activate Cloud Shell

Cloud Shell is a virtual machine that is loaded with development tools. It offers a persistent 5GB home directory and runs on the Google Cloud. Cloud Shell provides command-line access to your Google Cloud resources.

In the Cloud Console, in the top right toolbar, click the **Activate Cloud Shell** button.

Click **Continue**.



It takes a few moments to provision and connect to the environment. When you are connected, you are already authenticated, and the project is set to your *PROJECT_ID*. For example:

`gcloud` is the command-line tool for Google Cloud. It comes pre-installed on Cloud Shell and supports tab-completion.

You can list the active account name with this command:

```
gcloud auth list
```

(Output)

```
Credentialed accounts:
 - <myaccount>@<mydomain>.com (active)
```

(Example output)

```
Credentialed accounts:
 - google1623327_student@qwiklabs.net
```

You can list the project ID with this command:

```
gcloud config list project
```

(Output)

```
[core]
project = <project_ID>
```

(Example output)

```
[core]
project = qwiklabs-gcp-44776a13dea667a6
```

For full documentation of `gcloud` see the gcloud command-line tool overview.

Understanding Regions and Zones

Certain Compute Engine resources live in regions or zones. A region is a specific geographical location where you can run your resources. Each region has one or more zones. For example, the us-central1 region denotes a region in the Central United States that has zones `us-central1-a`, `us-central1-b`, `us-central1-c`, and `us-central1-f`.

Resources that live in a zone are referred to as zonal resources. Virtual machine Instances and persistent disks live in a zone. To attach a persistent disk to a virtual machine instance, both resources must be in the same zone. Similarly, if you want to assign a static IP address to an instance, the instance must be in the same region as the static IP.

Learn more about regions and zones and see a complete list in Regions & Zones documentation.

Task 1. Create a Vertex AI connection to enable ML calls using BigQuery SQL

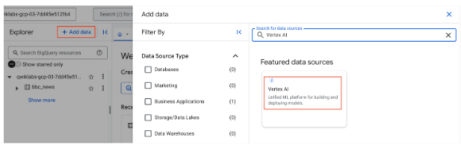
1. On the Google Cloud Console title bar, click the **Navigation Menu** (≡). Scroll to the Analytics section and click **BigQuery**.

2. In the Explorer pane of the BigQuery Console, click on the three dots next to your Project ID (`qwiklabs-gcp-01-21a89f2fd481`) and then select **Create dataset**.

3. In the Create dataset dialog, set the Dataset ID to `bbc_news` and make sure the multi-region `US` location is selected and leave the other options at their default values.

4. Click **Create dataset**.

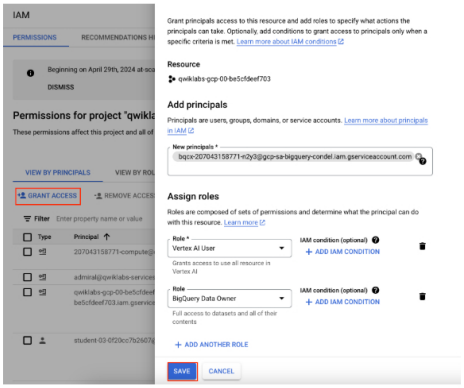
5. In the **Explorer** pane, click **+ Add data**, and then use the search bar for data sources to search for **Vertex AI**. Click on the result for **Vertex AI > Vertex AI Models: BigQuery Federation**.



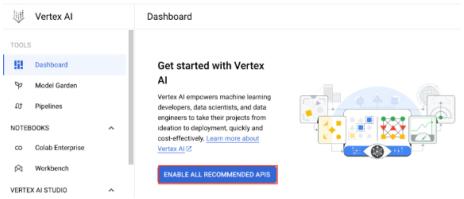
6. In the **Connection type** dropdown, select the `Vertex AI remote models, remote functions and BigLake (Cloud Resource)`. Set the Connection ID to `vertex-ai-connection`. Ensure the location is set to Multi-region `US`. Click the **Create Connection**.

7. In the **Explorer** pane, expand **External connections**, and select the `us.vertex-ai-connection` connection you just created. In the **Connection info** pane, copy the `Service account id` property value to the clipboard. You will need to add permissions to this account to use Vertex AI and your BigQuery data.

8. Click the **Navigation Menu** (≡) and go to **IAM & Admin** service. On the IAM page, click the **Grant Access** button. Paste the `service account id` into the `Principals` text box. Assign the **Vertex AI User** and **BigQuery Data Owner** roles to the account and click the **Save** button.



9. Click the **Navigation Menu** (≡) and navigate to the **Vertex AI** service. Click the **Enable All Recommended APIs** button.



10. Navigate to **BigQuery** service. In the BigQuery code editor, run the following command to create a model in your `bbc_news` dataset that connects to a Vertex AI Text Embedding model.

```
CREATE OR REPLACE MODEL `bbc_news.embedding_model`
  REMOTE WITH CONNECTION `us.vertex-ai-connection`
  OPTIONS (ENDPOINT = 'text-embedding-004');
```

11. In the Explorer pane, expand the `bbc_news` dataset and verify the model has been created.

Click **Check my progress** to verify the objective.

Create a Vertex AI connection

Check my progress

Task 2. Create Text Embeddings using BigQuery

In this task, you will create a table that contains articles from the Google BBC News public dataset as well as text embeddings that can be used for a vector similarity search.

1. Run the following SQL command to view some of the data. Notice the `title` and `body` fields contain the title and text from news articles.

```
SELECT * FROM `bigquery-public-data.bbc_news.fulltext` LIMIT 5;
```

2. Run the following SQL query. This creates a table named `bbc_news_with_embeddings_table` with the BBC News articles from the public dataset and text embeddings generated with the model named `embedding_model` you created above. Notice, that the title and body fields are concatenated into a new field called `content`. The function **ML.GENERATE_TEXT_EMBEDDING** looks for a field named `content` and generates the embeddings from that field.

Warning: This query will take 2 to 4 minutes to execute.

```
CREATE OR REPLACE TABLE
  `bbc_news.bbc_news_with_embeddings_table` AS (
  SELECT
    *
  FROM
    ML.GENERATE_TEXT_EMBEDDING( MODEL
`bbc_news.embedding_model`,
      (
      SELECT
        title,
        body,
        CONCAT(title, ": ", body) AS content
      FROM
        `bigquery-public-data.bbc_news.fulltext`
```

```
    WHERE
      LENGTH(body) > 0 AND LENGTH(title) > 0
    )
  ) WHERE ARRAY_LENGTH(text_embedding) > 0
  LIMIT 500
);
```

Note: If the table had more than 5000 rows, you would use the following command to create a Vector Index. This won't work in this example because the table has about 500 rows. The code below is shown here as a resource for implementing this pattern on larger projects, but don't run it as part of this lab.

```
CREATE OR REPLACE VECTOR INDEX bbc_news_index
ON `bbc_news.bbc_news_with_embeddings_table`(text_embedding)
OPTIONS(index_type = 'IVF',
  distance_type = 'COSINE',
  ivf_options = '{"num_lists":500}')
```

3. Expand the `bbc_news` dataset and select the `bbc_news_with_embeddings_table` table.

4. Take a look at the schema. Note that the `text_embedding` field has a type of `Float` with a mode of `Repeated`. This is how text embeddings should be represented in BigQuery, whether you generate them in BigQuery as we have done or if you generate them externally and upload them to a BigQuery table. Click on the **Preview** button to see a few rows of the data.

Click **Check my progress** to verify the objective.

Create Text Embeddings using BigQuery

Check my progress

Task 3. Run a Vector Search using BigQuery SQL

1. Now you will query the table for relevant news articles. Run the below query to return articles about `The US Economy`.

Note: An embedding is created for the string "The US Economy". That embedding is used in a similarity search and the rows for the top 5 articles with embeddings closest to the query's embedding are returned. Examine the results and verify the returned articles are related to the query.

```
SELECT query.query, base.title, base.body
FROM VECTOR_SEARCH(
  TABLE `bbc_news.bbc_news_with_embeddings_table`,
'text_embedding',
  (
  SELECT text_embedding, content AS query
  FROM ML.GENERATE_TEXT_EMBEDDING(
  MODEL `bbc_news.embedding_model`,
  (SELECT 'The US Economy' AS content))
  ),
  top_k => 5, options => '{"fraction_lists_to_search": 0.01}')
```

2. Change the search string and try a couple more queries. Search for things like: `"Political unrest in Africa"`, `"developments in the technology sector"`, and `"uplifting human success stories"`.

Click **Check my progress** to verify the objective.

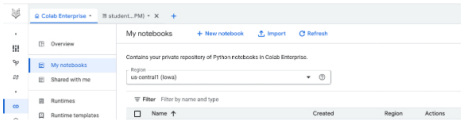
Run a Vector Search using BigQuery SQL

Check my progress

Task 4. Program a Q&A System using BigQuery and Gemini

1. Navigate to **Vertex AI** service and select **Colab Enterprise** from the Notebooks section of the navigation pane.

2. On **My Notebooks** pane, select the region as `us-west1` from dropdown and then click on **+ Create a New Notebook** button.



Enter the following code in the first cell and run it.

```
!pip install --upgrade --user google-cloud-aiplatform google-cloud-bigquery
```

It will take a few minutes for a runtime to start and the previous cell to run.

- While execution of above cell, you should be prompted with a new window **Open OAuth Popup**, then click on **OPEN** button.

- In next prompt choose an account of Qwiklabs student user (i.e. `student-02-5c8022007301`) to continue with **Colab Enterprise**.
- Once cell execution done, click on **RESTART SESSION**.

3. Once, the above cell execution done then add new cell. To add a new code cell click on **+ Code**.

- Paste the following code block. This code just sets variables for the project ID and location.

```
# Get project ID
PROJECT_ID = ! gcloud config get-value project
PROJECT_ID = PROJECT_ID[0]
LOCATION = "us-west1" # @param {type:"string"}
print(PROJECT_ID)
```

4. Add another code cell and paste the following code. It initializes Vertex AI.

```
from google.cloud import aiplatform
aiplatform.init(project=PROJECT_ID, location=LOCATION)

print("Initialized")
```

5. Add the following function in another code cell. This function allows you to pass a prompt to the Gemini LLM and get an answer.

```
import vertexai
from vertexai.generative_models import GenerativeModel, Part

def answer_question_gemini(prompt):
  model = GenerativeModel("gemini-pro")
  response = model.generate_content(
    prompt,
    generation_config={
        "max_output_tokens": 8192,
        "temperature": 0.5,
        "top_p": 0.5,
        "top_k": 10,
    },
  stream=False,
  )
  try:
    return response.text
  except:
    print("An Error Ocuured Cleaning the Data")
    return "An Error Ocuured Cleaning the Data"
```

6. Add the following function to another new code cell. This function takes as an argument the question the user asks, uses it to create an embedding, and uses a similarity search to find the top 5 articles most relevant to the question. After it runs the query, it concatenates the 5 five articles together into a single string and returns that string.

```
def run_search(question):
  from google.cloud import bigquery

  client = bigquery.Client()

  sql = """
      SELECT query.query, base.title, base.body
      FROM VECTOR_SEARCH(
      TABLE `bbc_news.bbc_news_with_embeddings_table`,
'text_embedding',
      (
      SELECT text_embedding, content AS query
      FROM ML.GENERATE_TEXT_EMBEDDING(MODEL
`bbc_news.embedding_model`,
          (SELECT @question AS content))),
      top_k => 5)
      """

  job_config = bigquery.QueryJobConfig(
    query_parameters=[
        bigquery.ScalarQueryParameter("question", "STRING",
question),
    ]
  )

  query_job = client.query(sql, job_config=job_config)

  data = ""
```

```
  for row in query_job:
    data += row.body + "\n"

  return data
```

7. Again, add another code cell and paste the following function. This function uses the question and the data retrieved from BigQuery to build a prompt that will be passed to the mode.

```
def build_prompt(data, question):
  prompt = """
    Instructions: Answer the question using the following
Context.

    Context: {0}

    Question: {1}
  """.format(data, question)
  return prompt
```

8. Add another code cell and paste the following function. This is helper function.

```
from IPython.core.display import display, HTML

def answer_question(question):

  data = run_search(question)
  display("Retrieved Data:")
  display(data)
  display(" . . . ")
  prompt = build_prompt(data, question)
  answer_gemini = answer_question_gemini(prompt)

  return answer_gemini
```

Click **Check my progress** to verify the objective.

Create a New Notebook

Check my progress

Task 5. Test the Q&A System

1. Add one more code cell to your notebook and enter the following code that asks a question and returns the answer.

```
QUESTION = "Tell me about the US Economy"

answer_gemini = answer_question(QUESTION)
display("User Question:")
display(QUESTION)
display("--------------------------------")
display("Gemini Answer:")
display(answer_gemini)
```

2. Try some other questions like, `"What's happening in sports?"`.

3. See what happens if you ask a question that has no relevant news articles. For example, `"What it a good recipe for bouillabaisse?"`.

Click **Check my progress** to verify the objective.

Test the Q&A System

Check my progress

Congratulations

You have now completed the lab! In this lab, you created text embeddings within BigQuery and used them in a RAG architecture to inform a language model's responses.

Next steps

- Check out the Generative AI on Vertex AI documentation.
- Learn more about Generative AI on the Google Cloud Tech YouTube channel.

Google Cloud Training & Certification

...helps you make the most of Google Cloud technologies. Our classes include technical skills and best practices to help you get up to speed quickly and continue your learning journey. We offer fundamental to advanced level training, with on-demand, live, and virtual options to suit your busy schedule. Certifications help you validate and prove your skill and expertise in Google Cloud technologies.

Manual Last Updated April 02, 2025

Lab Last Tested April 02, 2025